Programmable Object Placement Across Tiered Memory in OCaml 5

Andrew Li
Northwestern University
Evanston, IL, USA
andrewli@u.northwestern.edu

Abstract

Data-intensive workloads are increasingly bottlenecked on DRAM capacity, motivating heterogeneous memory hierarchies in which local DRAM is augmented by cheaper, slower "far memory" tiers such as CXL-attached memory. OS-level paging is blind to application object structure and places data of higher-level programming languages poorly. Recent work argues for *criticality*-based placement that accounts not just for hotness but for memory-level parallelism (MLP). Only the language runtime knows enough about object structure to make these decisions well. We present a framework for *programmable* garbage-collector object placement across tiered memory in OCaml 5. The runtime supplies all mechanism (arenas, promotion, pointer fixup, fact collection) as per the vanilla OCaml garbage collector; a user-supplied *policy*, a set of pure C functions, is dynamically loaded at runtime. The garbage collector backend supplies cheap facts about objects and asks the C library to decide the tier of memory for placement. We additionally thread `[@far_memory]`/`[@main_memory]` source attributes through the compiler so programmers can provide hints for their policies. We show that on an emulated CXL device, the framework adds little overhead over stock OCaml, and is a simple interface for researchers to prototype garbage collection policies without the need to recompile the entire language.

Keywords

OCaml, garbage collection, far memory, CXL, tiered memory, object placement

1 Background

Data-intensive workloads consume large quantities of DRAM, which is increasingly expensive given the growth of AI data centers. This memory bottleneck motivates a transition to heterogeneous memory hierarchies, where local DRAM is augmented by cheaper "far memory" tiers like Compute Express Link (CXL)-attached memory [2]. Traditional OS-level paging lacks insight into application object structure, leading to suboptimal placement of objects into pages across memory tiers. Heuristically it may seem reasonable to place objects by hotness or access frequency [6], but recent research has shifted toward criticality-based paging, which uses not just hotness but also memory-level parallelism (MLP). The intuition is that some data structures are friendly to hardware-level parallel access while others — pointer-chasing structures in particular — are not, so that the same access frequency can hide very different amounts of measured latency. The OS does not understand nor care about these notions of object accesss; only language runtimes do. Managed runtimes with garbage collection (GC) already relocate objects, and this object mobility is an advantage of managed over unmanaged languages [5]. OS-level frameworks such as PACT, with its Per-page Access Criticality metric [1] are an improvement over pure hotness-based heuristics, but are still blind to the runtime's structural knowledge.

1.1 Related work

PACT [1] argues for criticality-first tiered-memory placement but operates at the OS page level, blind to runtime object structure. TrackFM [4] provides compiler support for far memory over unmodified code and therefore works in the middle-end; our attributes are a more front-end contribution that lets the programmer state intent directly, similar to `[[likely]]` and `[[unlikely]]` from C++. Merchandiser [6] places data on heterogeneous memory by hotness and load balance. ALASKA [5] highlights object mobility as the lever managed runtimes have and unmanaged ones lack. We sit one layer below all of these: rather than ship a specific placement policy, we ship the runtime *mechanism* that lets a policy be written, swapped, and measured at near zero cost. The *handle* abstraction [5] gives unmanaged runtimes the same object-relocation freedom GCs have natively, and is directly relevant to future migration support in our framework.

1.2 Contribution

The contribution of this work is a **framework** and tiered design, not any one heuristic. The runtime owns the backend of the garbage collector — arena `mmap`-ing, underlying promotion mechanism, pointer fixup, thread coordination, bump allocation, and all of the work that existing garbage collectors do well. The policy is, in a sense, the frontend of the garbage collector, as it is the behavior that surfaces most directly to the developer of higher level software. The policy is completely agnostic of the garbage collector backend and receives metadata about objects that the backend knows to make a decision on which heap an object should be placed in. This is an increased expressiveness in policy as hypotheses can be tested by writing singular `.so` libraries in C to dynamically link into pre-compiled programs. Whether any particular policy wins is then an empirical question the framework lets us ask for cheaper than before.

We target OCaml 5 [3] because its multicore, concurrent GC makes the placement plumbing difficult, and because its generational so copying minor heap already promotes live objects to a major heap. That property of being generational creates an inherent tiered design that further tiering can naturally build upon.

Here, we introduce a place for further development that this work does not touch on in the slighest. The generational nature of OCaml 5's vanilla garbage collector is in essense a parent-child relationship. With the introduction of more memory tiers, this relationship between tiers can be expanded into entire family trees,

and objects being moved across heap boundaries can be expanded from minor-to-major to heap-to-heap. That may depend on object handles to abstract away addresses, which has been proven as a concept in prior work [5].

2 Design

2.1 Mechanism/Policy Split

Calls into a policy are unidirectional: the engine calls the policy. The policy is propagated to the engine as a a passive callback table of function pointers, each of which is a (usually) pure function that takes a metadata struct for the object in question and decides where to place it. The policy has no control over the backend. The significance of this design is that the policy is entirely sandboxed away from the garbage collector's backend. The rationale for why this is an improvement over tight coupling is that the policy is sandboxed from GC internals: it cannot allocate, cannot call back into the runtime, and the engine clamps its return value so a misbehaving policy degrades placement quality without threatening heap integrity. Purity also makes policies independently testable without a tight dependency on the OCaml GC backend.

Crucially, there is no byte serialization, no IPC, no shared memory, no ring buffer — no overhead other than loading the dynamic library using `dlopen`'d into the same address space. This is guaranteed by a single header, `runtime/caml/placement.h`. A one-time back-channel, `features_needed`, read once at `init`, lets a policy declare which non-free facts are worth computing, so policies that do not want an expensive fact never pay for it. At this stage of the project, none of those features are implemented, so this back-channel is a fancy nop.

2.2 The Policy Contract

The design of the policy, as hinted, can be described in a single sentence. The `choose_arena` function is pure

$$\texttt{choose\_arena} : \texttt{features} \longrightarrow \texttt{heap} \tag{1}$$

The intention is that the function is completely stateless. As each policy is its own translation unit, there is nothing fundamentally stopping a developer from encoding state as global variables, though that is not the intent of the design. There certainly exists policies for which global storage may be beneficial.

The benefits of purity are straightforward:

- **No concurrency protection.** While the OCaml 5 GC is parallel, if `choose_arena` is pure, then it will never race.
- **History for free.** A policy that wants aggregate history reads it from the `arena_used[]` snapshot the engine already places in `features`. Certainly a complex policy could want more detailed history, which is one instance where mutable state may be justified.

It was considered to create an OCaml API for the policy, but because the callback fires mid-collection while the heap is in an intermediate state (forwarding pointers, in-progress headers, the minor heap being drained). Running OCaml code at this stage would create a dependency cycle and could trigger a nested GC, which would corrupt the heap (it is unclear what exactly would happen). The callback is therefore **non-allocating C**, though it is possible to create higher-level DSLs that compile down to C, which could increase robustness of tracking state in mutable global variables.

2.3 Features

The GC backend fills a `caml_placement_features` record per object. There are facts stored here that are free because the GC already has them elsewhere or can trivially compute them, but either does not calculate them or does not have them in one place. Among these facts include the object's size in words (`wosize`), its `tag`, a `scannable` bit (1 = pointer-bearing, a free *low*-MLP proxy; 0 = flat/`No_scan`, a free *high*-MLP proxy), the compiler `hint` read from reserved header bits, the allocating domain, minor-heap occupancy, and the live-words-per-arena snapshot. Room for more expensive data is reserved, though are not implemented. If further data is to be added, the simple flat struct may need to be redesigned to align to cache lines, though this direction is unexplored.

Policies compiled in-between changes to the features struct may expect different struct sizes and offsets, which would create ABI problems. There can be improved input sanitisation done on the `.so` including encoding binary information in the data section, but this is not necessary for this proof of concept.

Table 1: Cheap, implemented fields of `caml_placement_features`

Field	Type	Description
`wosize`	`mlsize_t`	Size in words
`tag`	`tag_t`	Object tag (0–255)
`scannable`	`uint8_t`	1 = pointer-bearing (low MLP); 0 = flat (high MLP)
`hint`	`uint8_t`	Compiler attribute: `NONE` / `MAIN` / `FAR`
`site`	`uint8_t`	`PROMOTION` vs. `DIRECT`
`color`	`uint8_t`	GC color bits
`domain`	`int`	Allocating domain id
`minor_used`	`uintnat`	Words used in minor heap this cycle
`minor_size`	`uintnat`	Minor heap capacity in words
`arena_used[2]`	`uintnat[]`	Live words per arena (aggregate snapshot)

2.4 Registration and Loading

A single environment variable `CAML_GC_POLICY` is resolved at GC init before domains spawn. Values that could plausibly be paths to shared objects are `dlopen`'d and resolved, otherwise looked up in a built in table of default policies that are compiled into the runtime itself. The default is to fall back to `all_dram`, a single-arena policy that behaves exactly like stock OCaml. Crucially, the user's compiled OCaml programs are byte-for-byte identical regardless of policy: loading happens in the runtime at process start, decoupled from compilation.

2.5 Compiler Attributes

For manual placement we thread two source attributes, `[@far_memory]` and `[@main_memory]`, through the entire compiler pipeline. They are registered as known built-in attributes. The GC then reads the hint from the object header at promotion time. Policies are not required to abide by compiler hints, though doing so is a simple check of the `hint` feature, which incurs the cost of a single branch.

3 Deliverable

The deliverable is the contract and scaffolding, the infrastructure that `mmap`'s the non-DRAM heap, some simple policies, and the infrastructure that dynamically loads policies from `.so` files. A built-in policy is a few lines of C; a loadable one is the same plus an exported entry symbol.

4 Evaluation

We evaluate on a host with 800 GB Optane DAX (direct addressable) hardware which was shown through experimentation to be persistent, though that attribute is unused for the GC policies. We focus on four questions: do the tiers differ enough that the DAX memory acts as a proxy for CXL-based memory, how expensive is the framework, do policies affect performance, and what is the developer experience to implement a new policy.

4.1 Memory Tier Characterization

First, we verify that the two tiers are meaningfully different in latency and bandwidth. Figure 1 shows bandwidth and latency for sequential and random access patterns on DRAM and the Optane DAX partition used as the far tier. The DAX tier achieves substantially lower peak bandwidth and higher read latency than local DRAM, consistent with published CXL-attached memory profiles [2], justifying its use as a CXL proxy.

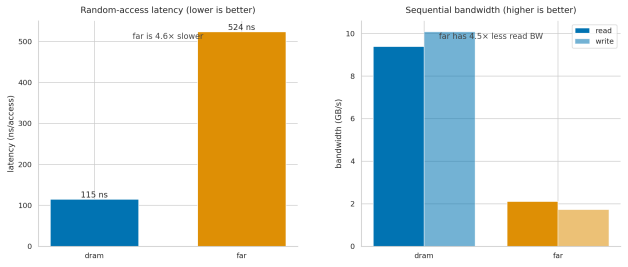

Figure 1: Bandwidth and latency profiles of DRAM and Optane DAX. The DAX tier's lower bandwidth and higher latency make it a reasonable proxy for CXL-attached memory.

4.2 Discussion

The hardware results (Figure 1) confirm that placement is worthwhile: the tiers differ enough that routing the wrong objects to far memory carries a real cost. The overhead is demonstrably small at less than 1 ns (Figure 2). For larger programs, as the shared library is loaded once, that cost is effectively zero. These two observations establish that the framework is a useful substrate for GC policy research.

Note that GC policy is here used to refer to *promotion* policy, which is an entire novel field compared to eviction policy or underlying algorithm. The lack of tiers in current memory means that GCs fundamentally do not have policies that make decisions across heterogenous heaps.

Policy effectiveness is workload-dependent, as expected (Figure 3). The `scannable` bit is a coarse MLP proxy: it captures the

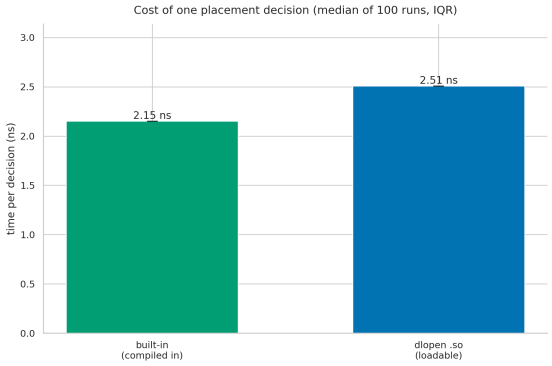

Figure 2: Per-decision dispatch overhead of the placement framework relative to stock OCaml. The `dlopen` path adds no distinguishable cost over the built-in path.

structural distinction between pointer-chasing and flat data, but says nothing about access frequency or fanout depth within those classes. A linked list of 1000 nodes and a single-element wrapper record are both scannable, yet their MLP profiles differ entirely. This is the natural limit of the cheap features used and motivates the implementation of more expensive features such as `chain_depth`.

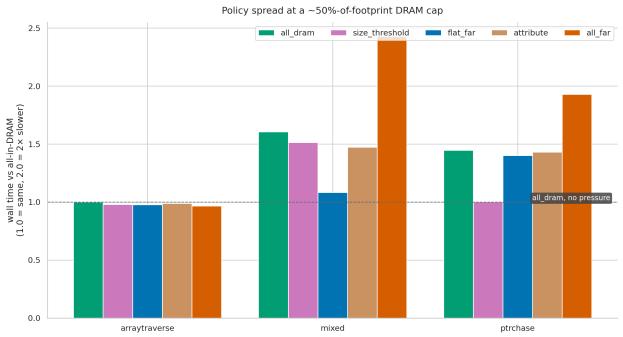

Figure 3: Wall-clock runtime under `all_dram`, `all_far`, and `flat_far` placement policies. A Tier-0 policy that routes flat objects to far memory recovers measurable performance over the all-far control on pointer-chasing workloads.

The compiler attributes address a complementary axis. A heuristic operates on structural facts visible at promotion time; an attribute encodes programmer intent at allocation time. The two can be composed: a policy that first checks `hint` and falls back to the `scannable` heuristic for unannotated objects gets both benefits. Because policies are independent C translation units, this composition requires no changes to the runtime. Because hints are not strict, policy developers can ignore them freely, and can decide how to counterbalance hints and features.

The developer experience result is perhaps the most practical finding. A new placement hypothesis that previously required recompiling the OCaml runtime now requires writing a handful of

C lines and setting an environment variable (Figure 4). This lowered barrier directly enables empirical GC research: hypotheses can be tested, compared, and discarded at a rate the prior approach could not support. It also opens the door to LLM-assisted policy generation, where a language model proposes placement heuristics expressed over the `caml_placement_features` struct without any knowledge of the GC internals, which increases the efficacy of large context windows to be efficiently used instead of wasted on the underlying backend code that does not affect a GC policy.

A funny parallel to high-frequency trading[1] can be drawn. The market and exchange are analogous to the GC backend and while we care about the data, we don't necessarily care about how it works — just that it does. We receive data from these institutions on feeds taking known binary formats, just as the GC backend passes data to the policy via a struct. And our policy that decides where an obejct should be placed is analogous to the strategy that algorithms use to decide what to trade and how much. The reason we draw this parallel is to make the following three arguments:

(1) ML model based GC. Trading strategies are frequently based on deep learning models that are trained on a variety of inputs. Garbage collectors could be trained on models that take heap state as input (similar to a market where we have snapshots of orderbooks, in a GC we can have snapshots of the GC internals) and return a prediction for the most optimal placement as output. The issue with this is very high prediction latency, not prediction strength.
(2) 'Signal' based GC. Trading strategies are sometimes composed of linear combinations of a variety of signals ('features'). These are identical in architecture to the features passed to the GC. The design was somewhat inspired by the architecture from trading, as these signals are composed together similarly GC policies for this project with simple functions that are abstracted away from the GC/market internals.
(3) The difference between trading and GC placement is that in trading, a strategy can decide to not make a trade. In a GC, the policy cannot decide to not place an object on a heap. While it can be argued that placement in DRAM can be a sensible default, that is still fundamentally not a no-op operation that *not* placing an order to an exchange is.

ML/Signal based GC is a interesting direction for future work. The complexity here is that OCaml is overly complex as a harness, but that may make the research more interesting and worthwhile. The compute-boundedness of the direction is glaringly obvious, but positive results could motivate further research into increasing the speed of embedded machine learning models. The tension is between high-latency predictions and possible performance improvements obtained from more optimal object placement.

A current limitation is that only static placement is evaluated. An object's access pattern can shift after promotion: a working-set object initially accessed frequently may become cold, while a previously cold data structure may be pulled into a hot loop. Far-to-DRAM *migration* after promotion would address this, but requires

[1] Might be because I saw this part of HRT's codebase 6 hours before writing this section haha

```c
static int choose_arena(const caml_placement_features *f)
{
  return f->arena_used[CAML_ARENA_DRAM] < budget_words
       ? CAML_ARENA_DRAM : CAML_ARENA_FAR;
}
```

```c
static int choose_arena(const caml_placement_features *f)
{
  /*-1. An explicit programmer hint always wins. */
  if (f->hint == CAML_HINT_FAR)  return CAML_ARENA_FAR;
  if (f->hint == CAML_HINT_MAIN) return CAML_ARENA_DRAM;

  /*-2. Code and continuations are latency-critical — never banish them. */
  if (f->tag == Closure_tag || f->tag == Infix_tag || f->tag == Cont_tag)
    return CAML_ARENA_DRAM;

  /*-3. Flat bulk (strings, doubles, custom blocks) is bandwidth-friendly and
       rarely pointer-chased; once bigger than a cache line, send it far. */
  if (!f->scannable && f->wosize >= 8)
    return CAML_ARENA_FAR;

  /*-4. Small pointer structures: keep it local while DRAM has headroom relative
       to far (the free-occupancy snapshot as a soft 3:1 cap), but bias toward
       far when this domain's minor heap is already crowded — a burst of
       promotions is about to land. */
  uintnat dram = f->arena_used[CAML_ARENA_DRAM];
  uintnat far  = f->arena_used[CAML_ARENA_FAR];
  int dram_has_room = dram < 3 * (far + 1);
  int minor_crowded = (f->minor_used * 4) > (f->minor_size * 3);

  if (!dram_has_room) return CAML_ARENA_FAR;
  if (minor_crowded && f->site == CAML_PLACE_PROMOTION) return CAML_ARENA_FAR;
  return CAML_ARENA_DRAM;
}
```

Figure 4: Left: a trivial all-DRAM policy. Right: a more structured policy that weighs the compiler hint and arena fill level. Both are self-contained C translation units under 30 lines.

whole-heap pointer fixup, equivalent to compaction. Object handles [5] would make this cheaper, and the `should_migrate` callback slot in the policy vtable is reserved for this purpose. We leave it as future work.

5 Conclusion

Managed runtimes have a structural advantage over unmanaged code in a tiered-memory world: the GC already relocates objects. OCaml 5 already promotes objects and it was trivial to `mmap` a second arena. Placement policy is difficult to engineer, as the intuitive option is to write it somewhere deep within the GC because it's unlikely the average developer will want to write custom GC policies. By pulling placement into its own software layer, ordinary developers can write placement policies that are specially tailored to their programs. Evaluation on 800 GB Optane DAX hardware shows that the overhead of dynamically loadable policies is under 1 ns.

The contribution is the research infrastructure. A placement hypothesis that previously required recompiling the OCaml runtime now costs a handful of C lines and an environment variable. The same interface is amenable to LLM-assisted policy generation and to signal-based policies inspired by quantitative finance, where linear combinations of object features (size, tag, depth, hotness) serve the same role as alpha signals in a trading strategy.

Several directions remain open. More convoluted features can be implemented and instrumented. Object handles [5] would unlock post-promotion migration without full-heap compaction. A higher-level policy DSL, or a machine-learned policy trained on heap snapshots, could be layered on the existing C interface without any runtime change.

The most poignant direction for future work is to experiment with what policies are best for which workloads, and whether it makes sense in a production language to expect developers to write custom GC policies. The argument can be made that once a developer wants to control memory at such a granular level, they may as well just use C++. Some environments could be locked into specific programming languages, and increasing control of managed programming languages gives those developers more freedom. The conclusion is not a positive result, but establishes infrastructure to find positive results.

A second direction for immediate future work accounts for the size of the DAX-based device. That device has 800 GB of addressable

space, which implies that putting aside the tiered memory in GCs, even just porting a language to only use DAX-based memory in place of DRAM could mean fewer pages swapped. That would increase performance.

References

[1] Hamid Hadian, Jinshu Liu, Hanchen Xu, Hansen Idden, and Huaicheng Li. 2026. PACT: A Criticality-First Design for Tiered Memory. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. ACM, Pittsburgh, PA, USA. doi:10.1145/3779212.3790198

[2] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *Comput. Surveys* 56, 11, Article 290 (July 2024), 37 pages. doi:10.1145/3669900

[3] KC Sivaramakrishnan, Stephen Dolan, Leo White, Sadiq Jaffer, Tom Kelly, Anmol Sahoo, Sudha Parimala, Atul Dhiman, and Anil Madhavapeddy. 2020. Retrofitting Parallelism onto OCaml. *Proceedings of the ACM on Programming Languages* 4, ICFP, Article 113 (Aug. 2020), 30 pages. doi:10.1145/3408995

[4] Brian R. Tauro, Brian Suchy, Simone Campanoni, Peter Dinda, and Kyle C. Hale. 2024. TrackFM: Far-out Compiler Support for a Far Memory World. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '24)*. ACM, La Jolla, CA, USA. doi:10.1145/3617232.3624856

[5] Nick Wanninger, Tommy McMichen, Simone Campanoni, and Peter Dinda. 2024. Getting a Handle on Unmanaged Memory. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*. ACM, La Jolla, CA, USA. doi:10.1145/3620666.3651326

[6] Zhen Xie, Jie Liu, Jiajia Li, and Dong Li. 2023. Merchandiser: Data Placement on Heterogeneous Memory for Task-Parallel HPC Applications with Load-Balance Awareness. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*. ACM, Montreal, QC, Canada. doi:10.1145/3572848.3577497

A AI Usage

AI enabled me to take on a more feature-based, "PM-like" role — more concretely, AI was the code writer for this project. The observations from using AI for a large scale project are the same as those I've had working in industry: that working with AI is split into three phases. Gathering context, refining a plan, and executing that plan. From some of the logs, it can be seen that I spent a lot of time making sure the AI knew what files to read, what was going on, what the vision was, etc. Then, I would let it take free reign and write a rough draft of a plan of what it *wanted* to do. The way that software is changing is that we don't necessarily care about specific code being written, but the level of detail in the plans we give to AI. The second phase is refining the plan, and this may entail overt criticality of AI's word choice, phrasing, what it chooses to focus on, etc. We as the technical overseers know what we want, and while sometimes AI gives us useful answers, ultimately we are the ones making the decisions. Sometimes, when we don't know, we may ask AI to decide for us. Usually, this is a mistake. The better solution is to ideate with AI in a conversation style, such that we maintain the unilateral decision making capability. Only once we make the plan as concrete as possible should AI be allowed to execute.

In a first run through of much of this project, I did not make the plans *as concrete as possible*. This led to the code being implemented to be sloppy and monolithic, whereas in the final run through that was presented, the spec was agreed upon before a single line of code was written. This is the way things should be. This is the reason that some companies require developers to write extensively and unnecessarily long design docs before handing the rest off to Claude. Not because a human will have doubt after the first 10, but because we don't want AI to have any confusion.

The engineering from Claude was mediocre, to be frank. The code that was written was only good after instructing Claude to maintain high code quality standards, use sane naming, spend time thinking about names, etc. The default is to vomit everything out without even adding newlines between sections of code.

If I wrote the code by hand, I would definitely be more intimately familiar with OCaml's APIs. Using AI gave me much more useful architectural level insight, which is more applicable. The applicability is because the architecture can be used pretty much anywhere, while specific lines of code are pretty useless outside of the classroom and school environment. So intro classes may not benefit much using tools like Claude as 300 and 400 levels.

The paid plan was good but ultimately still had rate limits. The industry norm is to use API billing which gets expensive quickly (I've spent probably >$300 in a single day, which can be on the low side at some companies). Ultimately the use of funds was good, but still not reflective of industry practices.

One more insight both from working here and in industry is splitting work across sessions. Once sessions get long, context cache lookups get VERY expensive (I had to use my own account, and it DRAINED my credits). Splitting these sessions into multiple and sharing context via Markdown files exponentially cuts the rate at which money is spent.

One side of me thinks I should have written more code by hand for this project, but then I would have less opportunity to explore higher level architectural ideas. There is a balance to strike. At work, I'm starting to write more specs by hand before handing off to Claude, specifically where I know Claude will get things wrong.

As a funny example from the insights document:

> Asked to 'make the text white,' Claude enthusiastically built an entire env-gated dark-mode feature before the user interrupted to explain they just wanted... white text.

To be specific and communicate ideas clearly is the challenge for the next generation of computer scientists, not writing code. For this reason and others, I regret not doing a minor in comparative literature.

B Code Availability

Code can be found at `https://github.com/andrlime/ocaml`. The plan used to guide Claude Code can be found at `PLAN.md`. There are some unimplemented stretch goals described in that document.

C Final Thoughts

I should probably clean this up. I think the ideas are really cool. But everything is all over the place... maybe that's a consequence of using Claude.